



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Natalia Siddharta

NIM : 2428240096

Kelas : SI5C

**PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026**

Identitas Praktikum

Nama Mahasiswa	Natalia Siddharta
NPM	2428240096
Kelas	SI5C
Pertemuan Ke-	Pertemuan 3 dan Pertemuan 4
Topik / Judul Praktikum	Tugas 1 – RESTful API Murni dengan Express.js (Topik 4: Mata Kuliah / Courses)
Tanggal Pelaksanaan	28 September 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Membangun RESTful API murni dengan Express.js yang hanya mengembalikan response berformat JSON (tanpa tampilan/HTML) untuk mengelola resource mata kuliah pada endpoint /courses.
2. Menerapkan operasi CRUD lengkap menggunakan method GET, POST, PUT, dan DELETE, serta filter data dengan query string (req.query) dan route parameter :id (req.params.id).
3. Menerapkan validasi field wajib, format response {status, message, data}, middleware catch-all 404, dan penggunaan status code HTTP yang tepat (200, 201, 400, 404).
4. Menguji seluruh endpoint menggunakan Postman, mengelola kode secara bertahap dengan Git dan GitHub, serta melakukan deployment aplikasi ke Vercel.

II. Dasar Teori

RESTful API. REST (Representational State Transfer) adalah gaya arsitektur perangkat lunak terdistribusi yang diperkenalkan oleh Fielding (2000). Pada REST, setiap data dipandang sebagai *resource* yang diidentifikasi oleh URL, sedangkan operasi terhadap resource dinyatakan melalui method HTTP. RESTful API adalah layanan web yang menerapkan prinsip tersebut dan membalas permintaan dalam format data, umumnya JSON, bukan halaman web. Nama resource berupa kata benda jamak (misalnya /courses) dan tidak memuat kata kerja pada URL.

Method dan status code HTTP. Method GET digunakan untuk membaca data, POST untuk menambah data baru, PUT untuk mengganti seluruh data pada resource tertentu, dan DELETE untuk menghapus data (Fielding et al., 2022). Hasil pemrosesan dinyatakan dengan status code: 200 (OK) untuk permintaan yang berhasil, 201 (Created) ketika data baru berhasil dibuat, 400 (Bad Request) ketika permintaan tidak valid, dan 404 (Not Found) ketika resource tidak ditemukan (MDN Web Docs, n.d.).

Express.js. Express.js adalah framework web minimalis untuk Node.js yang menyediakan routing (app.get, app.post, app.put, app.delete) dan middleware (Express, n.d.). Middleware `express.json()` membaca body JSON ke dalam `req.body`, `req.params` menyimpan route parameter seperti `:id`, dan `req.query` menyimpan query string seperti `?semester=3`. Middleware catch-all yang dipasang setelah semua route digunakan untuk menangani endpoint yang tidak terdaftar.

Postman dan Vercel. Postman digunakan untuk mengirim request dan memeriksa response serta status code (Postman, n.d.). Vercel adalah platform cloud untuk deployment yang menjalankan aplikasi Express sebagai serverless function, sehingga aplikasi perlu diekspor dengan `module.exports = app` (Vercel, n.d.).

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Versi LTS; runtime JavaScript
2	Visual Studio Code	Editor kode
3	Express.js	Versi ^5.2.1; framework web (dependency)
4	nodemon	Versi ^3.1.14; menjalankan ulang server otomatis (devDependency)
5	Postman	Pengujian endpoint API
6	Git dan GitHub	Version control dan repository publik tugas1-restful-2428240096
7	Vercel	Platform deployment (serverless)
8	Google Chrome	Memeriksa hasil GET pada URL deployment

IV. Langkah Kerja

1. Membuat folder project tugas1-restful-2428240096, lalu menginisialisasi project dengan `npm init -y`.
2. Memasang Express sebagai dependency (`npm install express`) dan nodemon sebagai devDependency (`npm install -D nodemon`).
3. Mengatur package.json: "main": "app.js", script "start": "node app.js", dan script "dev": "nodemon app.js".
4. Membuat .gitignore (berisi node_modules/) dan vercel.json, kemudian menjalankan git init dan commit pertama dengan pesan "first commit".

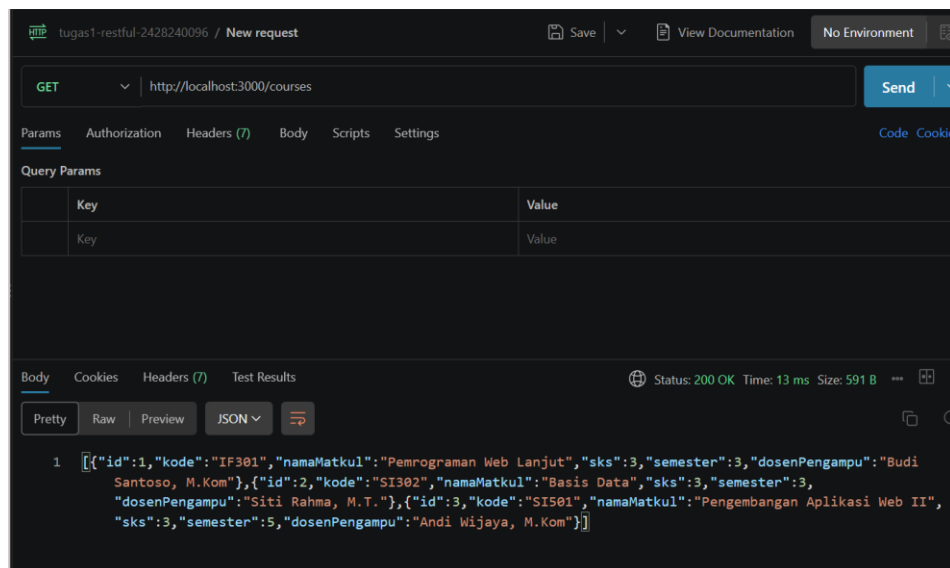
5. Membuat app.js: mengimpor Express, memasang `express.json()`, membuat array `courses` (3 data awal), variabel `nextId`, fungsi validasi, dan route GET / yang berisi info API. Lalu commit.
6. Menambahkan route GET, yaitu GET /courses (termasuk filter `?semester=`) dan GET /courses/:id dengan respons 404 bila id tidak ada. Lalu commit.
7. Menambahkan route POST /courses yang memvalidasi field wajib (400 bila tidak lengkap), membuat id otomatis dengan `nextId++`, dan membalas 201. Lalu commit.
8. Menambahkan route PUT /courses/:id yang memeriksa keberadaan data (404), memvalidasi body (400), lalu mengganti seluruh field data. Lalu commit.
9. Menambahkan route DELETE /courses/:id (404 bila tidak ada), middleware catch-all 404, dan penanganan error JSON. Menyesuaikan bagian bawah app.js untuk Vercel (`app.listen` hanya dijalankan jika `NODE_ENV !== "production"` dan `module.exports = app`). Lalu commit.
10. Menjalankan server dengan `npm run dev`, lalu menguji seluruh endpoint di Postman pada `http://localhost:3000` (untuk POST dan PUT: Body → raw → JSON) dan mengambil screenshot tiap pengujian.
11. Membuat repository GitHub publik `tugas1-restful-2428240096`, menghubungkannya dengan `git remote add origin`, lalu `git push -u origin main`.
12. Meng-import repository ke Vercel, menekan Deploy, kemudian menguji endpoint pada URL deployment `https://tugas1-restful-2428240096.vercel.app`.

V. Hasil dan Pembahasan

Pengujian dilakukan menggunakan Postman pada server lokal (`http://localhost:3000`), kemudian hasilnya diperiksa pada POSTMAN. Data awal berisi tiga mata kuliah (id 1 sampai 3), sehingga data baru dari POST memperoleh id 4. Ringkasan hasil pengujian ditunjukkan pada Tabel 1, dan tangkapan layar tiap pengujian ditunjukkan pada Gambar 1 sampai Gambar 13.

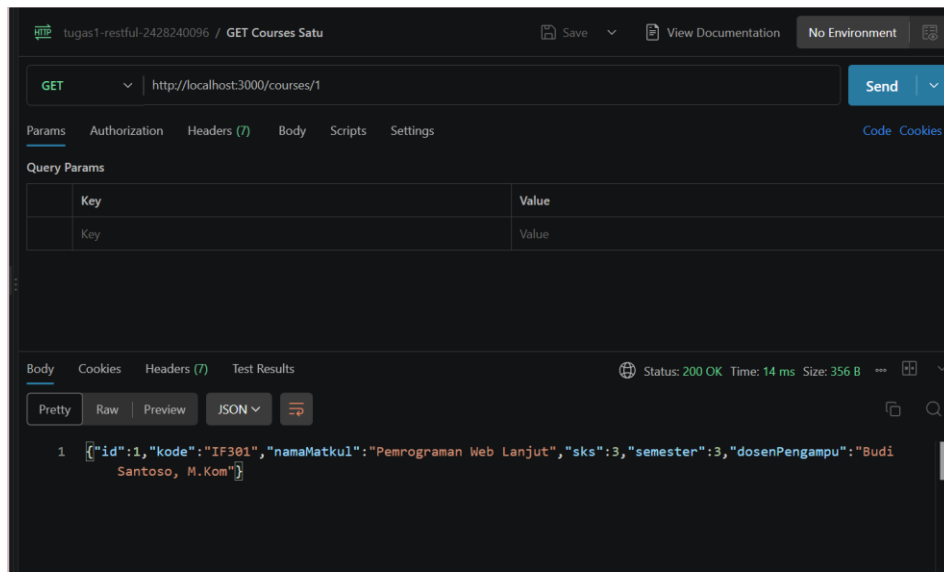
Tabel 1. Hasil pengujian endpoint pada server lokal

N o	Metho d	Endpoint	Data request	Status diha rkan	Status hasil	Keterangan
1	GET	/courses	—	200	200	Sesuai
2	GET	/courses/1	—	200	200	Sesuai
3	GET	/courses/99	—	404	404	Sesuai
4	GET	/courses?semester=3	—	200	200	Sesuai
5	POST	/courses	body lengkap	201	201	Sesuai
6	POST	/courses	field wajib kosong	400	400	Sesuai
7	PUT	/courses/1	body lengkap	200	200	Sesuai
8	PUT	/courses/99	id tidak ada	404	404	Sesuai
9	DELETE	/courses/1	—	200	200	Sesuai
10	DELETE	/courses/99	id tidak ada	404	404	Sesuai
11	GET	/abc	—	404	404	Sesuai



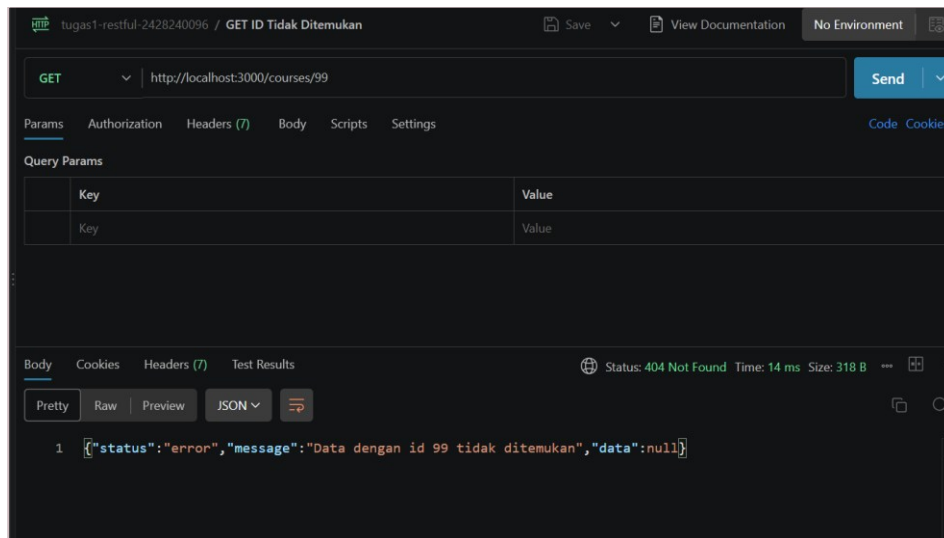
Gambar 1. Pengujian GET /courses (ambil semua data) – 200 OK

GET dipilih karena hanya membaca data dan tidak mengubah array. Response berupa array berisi tiga mata kuliah yang dikembalikan langsung, tanpa status dan message, dengan status 200 OK sesuai ketentuan.



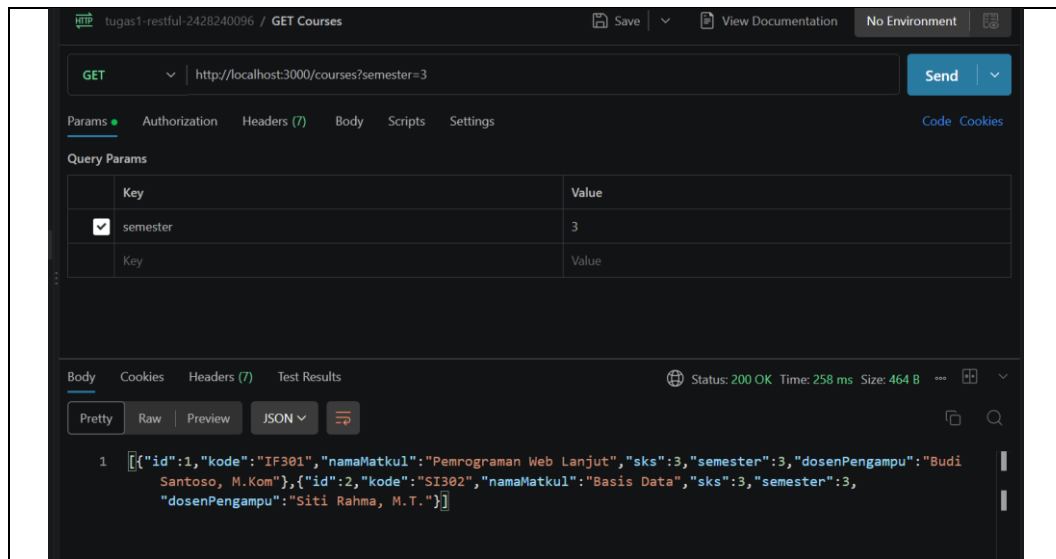
Gambar 2. Pengujian GET /courses/1 (ambil satu data) – 200 OK

Route parameter :id dibaca dengan req.params.id, kemudian data dicari menggunakan find(). Karena data dengan id 1 ada, server mengembalikan satu objek (IF301 – Pemrograman Web Lanjut) dengan status 200.



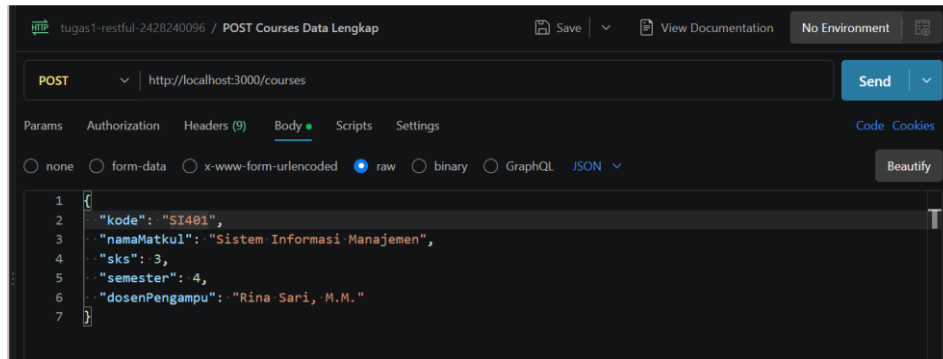
Gambar 3. Pengujian GET /courses/99 (id tidak ditemukan) – 404 Not Found

Id 99 tidak ada di array, sehingga server membalas 404 dengan format { status: "error", message: "Data dengan id 99 tidak ditemukan", data: null }.



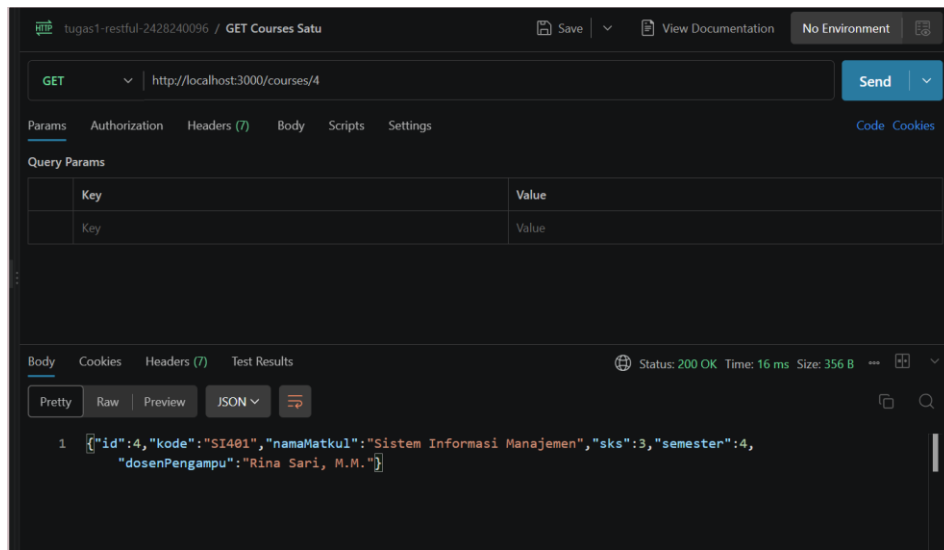
Gambar 4. Pengujian filter GET /courses?semester=3 – 200 OK

Filter menggunakan query string (`req.query.semester`), bukan route parameter, sesuai ketentuan tugas. Nilainya diubah menjadi angka dengan `parseInt`, lalu array disaring dengan `filter()` dan dikembalikan langsung sebagai array (boleh kosong) dengan status 200.



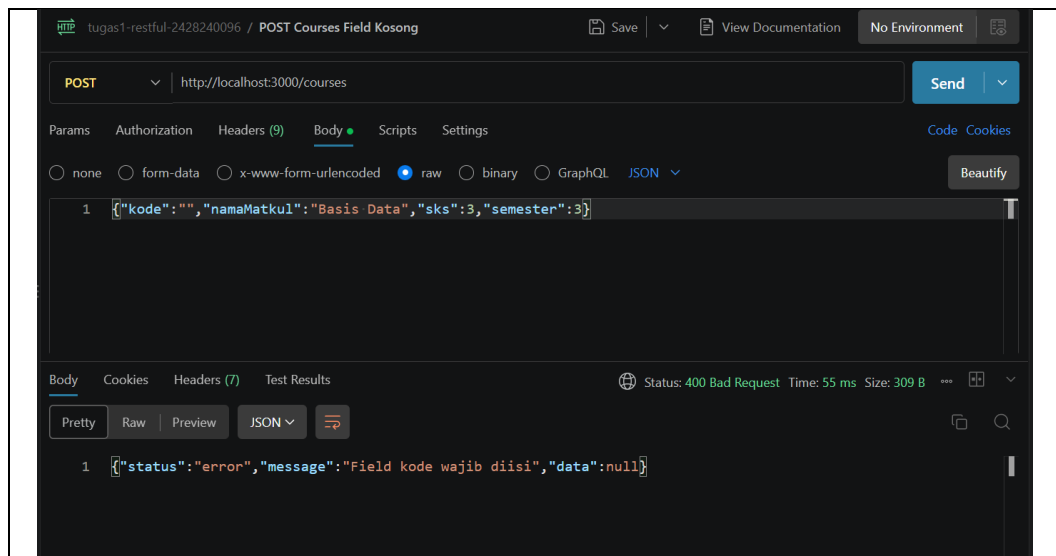
Gambar 5. Request POST /courses dengan body JSON lengkap

POST dipilih karena bertujuan menambah data baru. Body JSON (`kode`, `namaMatkul`, `sks`, `semester`, `dosenPengampu`) dibaca dengan `express.json()`. Id dibuat otomatis oleh server (`nextId++`) dan tidak dikirim di body. Jika validasi lolos, server membalas 201 Created beserta data yang baru dibuat.



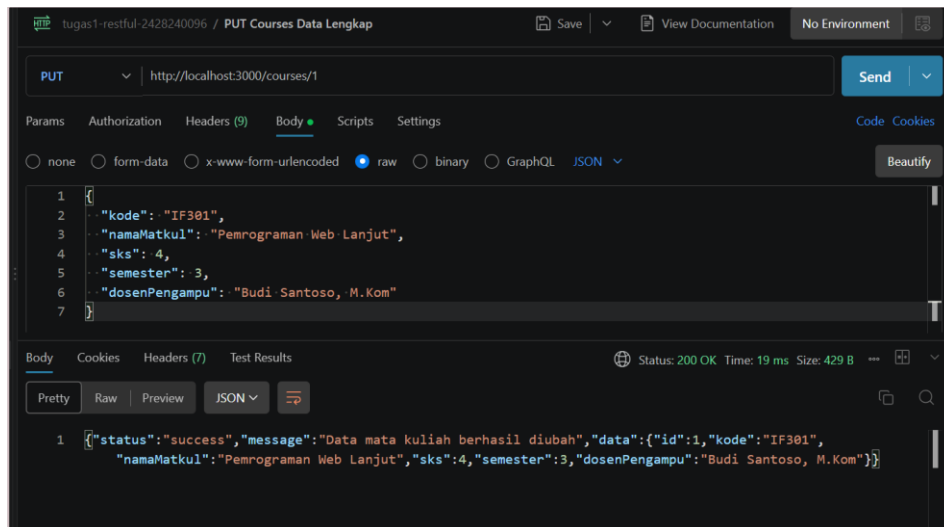
Gambar 6. Verifikasi hasil POST: GET /courses/4 menampilkan data baru – 200 OK

Data baru tersimpan dengan id 4 (id terakhir + 1) dan dapat diambil kembali melalui GET, yang menunjukkan proses POST berhasil.



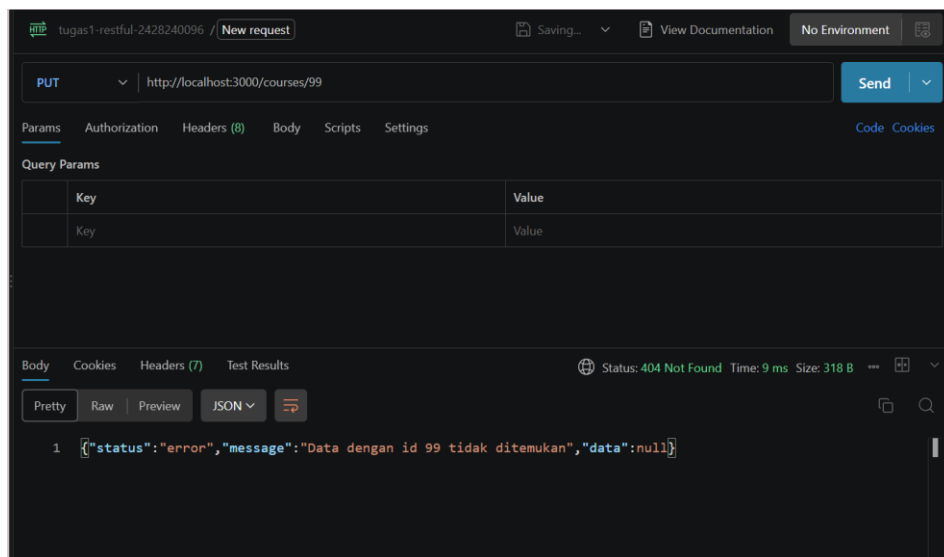
Gambar 7. Pengujian POST /courses dengan field wajib kosong – 400 Bad Request

Fungsi validasi memeriksa kode, namaMatkul, sks, dan semester sebelum data disimpan. Jika ada yang kosong (atau sks/semester bukan angka), server membalas 400 Bad Request dengan pesan “Field ... wajib diisi” dan data: null, sehingga array tidak berubah.



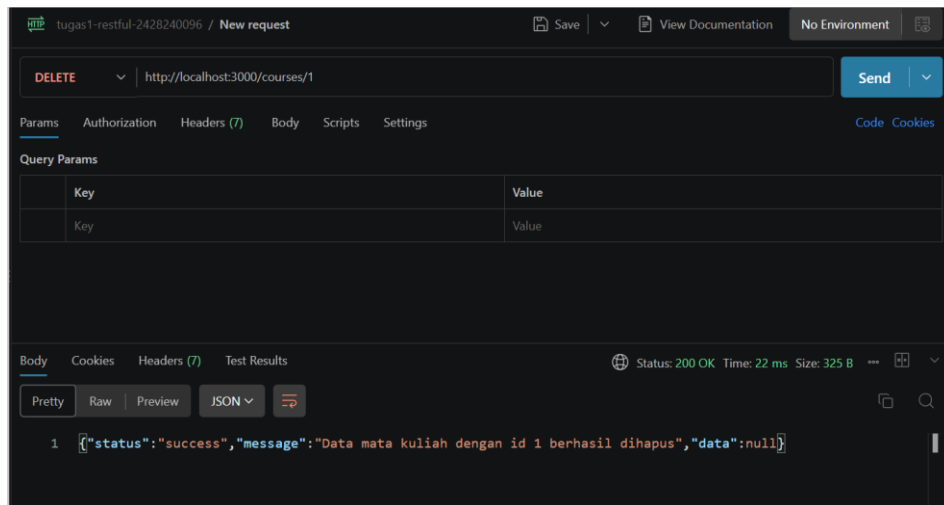
Gambar 8. Pengujian PUT /courses/1 dengan body lengkap – 200 OK

PUT dipilih karena mengganti seluruh field pada data dengan id tertentu (penggantian penuh). Pada pengujian, nilai sks diubah dari 3 menjadi 4, dan response menampilkan data terbaru dengan pesan “Data mata kuliah berhasil diubah”.



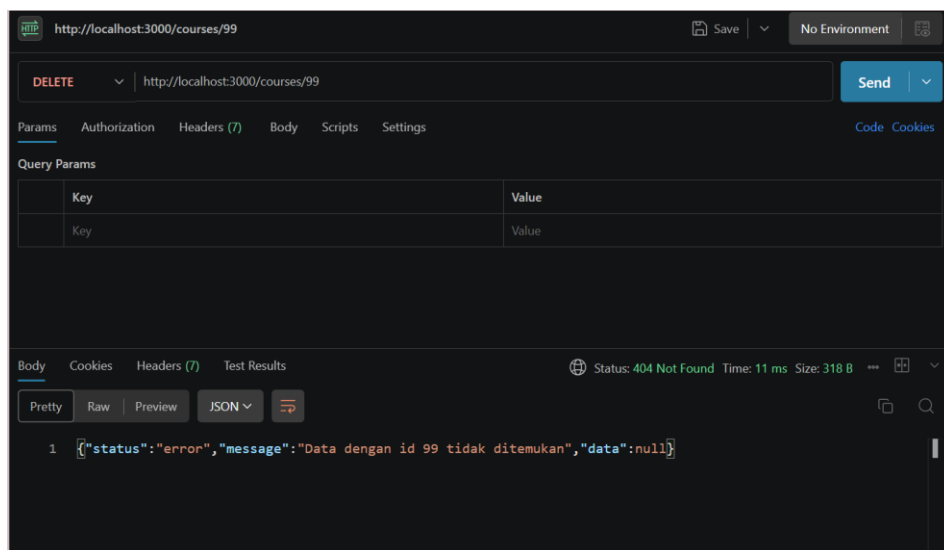
Gambar 9. Pengujian PUT /courses/99 (id tidak ditemukan) – 404 Not Found

Server memeriksa keberadaan data terlebih dahulu sebelum memvalidasi body. Karena id 99 tidak ada, server membalas 404 dengan data: null dan array tidak berubah.



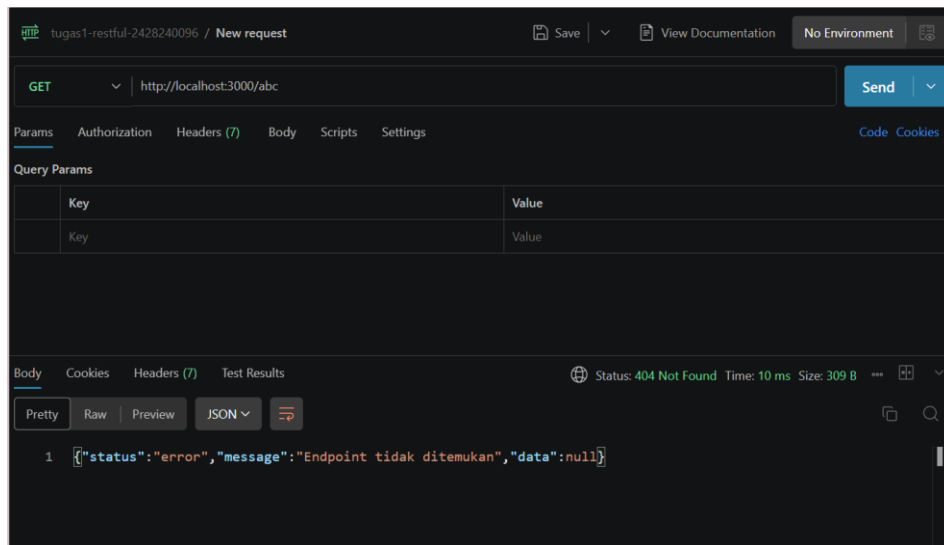
Gambar 10. Pengujian DELETE /courses/1 – 200 OK

DELETE menghapus data dengan splice(). Response berstatus 200 dengan data: null dan pesan yang menyebut id yang dihapus (“Data mata kuliah dengan id 1 berhasil dihapus”).



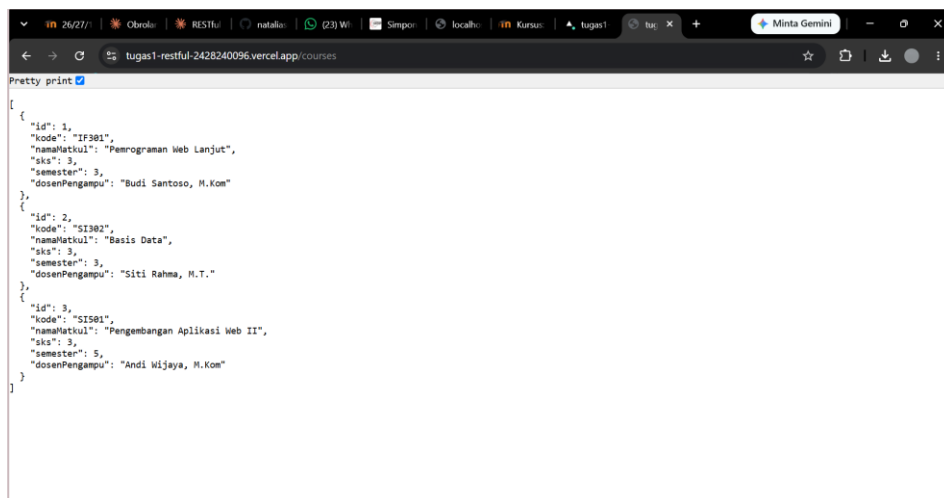
Gambar 11. Pengujian DELETE /courses/99 (id tidak ditemukan) – 404 Not Found

Karena id 99 tidak ditemukan oleh findIndex() (hasil -1), penghapusan ditolak dengan status 404 dan format error yang sama seperti endpoint lainnya.



Gambar 12. Pengujian route tidak terdaftar (GET /abc) – 404 Not Found

Middleware catch-all yang dipasang setelah semua route menangkap endpoint yang tidak dikenal dan tetap membalas JSON “Endpoint tidak ditemukan”, bukan halaman HTML bawaan Express.



Gambar 13. Hasil DATA courses pada URL deployment Vercel

Setelah repository di-import ke Vercel, endpoint dapat diakses publik pada `https://tugas1-restful-2428240096.vercel.app/courses` dan mengembalikan tiga data awal dalam format JSON. Agar dapat berjalan di Vercel, bagian bawah `app.js` disesuaikan: `app.listen` hanya dijalankan di luar production dan aplikasi diekspor dengan `module.exports = app`.

Tabel 2. URL pengujian pada deployment Vercel

No	Method	URL lengkap	Harapan
1	GET	https://tugas1-restful-2428240096.vercel.app/courses	200
2	GET	https://tugas1-restful-2428240096.vercel.app/courses/1	200
3	GET	https://tugas1-restful-2428240096.vercel.app/courses/99	404
4	GET	https://tugas1-restful-2428240096.vercel.app/courses?semester=3	200
5	POST	https://tugas1-restful-2428240096.vercel.app/courses	201
6	POST	https://tugas1-restful-2428240096.vercel.app/courses	400
7	PUT	https://tugas1-restful-2428240096.vercel.app/courses/1	200
8	PUT	https://tugas1-restful-2428240096.vercel.app/courses/99	404
9	DELETE	https://tugas1-restful-2428240096.vercel.app/courses/1	200
10	DELETE	https://tugas1-restful-2428240096.vercel.app/courses/99	404

Pembahasan:**1. Pemilihan method**

Tiap method dipilih sesuai fungsinya: GET hanya membaca sehingga tidak mengubah array; POST menambah data baru dengan id otomatis; PUT mengganti seluruh field sehingga semua field wajib dikirim ulang; DELETE menghapus data berdasarkan id. Route parameter hanya dipakai untuk :id, sedangkan filter memakai query string (?semester=3).

2. Pemilihan status code

Status 200 dipakai untuk GET, PUT, dan DELETE yang berhasil; 201 untuk POST karena membuat data baru; 400 ketika field wajib tidak lengkap atau tipe data salah; dan 404 ketika id atau endpoint tidak ditemukan. Dengan status code yang tepat, klien dapat membedakan hasil sukses dan kesalahan tanpa membaca isi pesan.

3. Format response

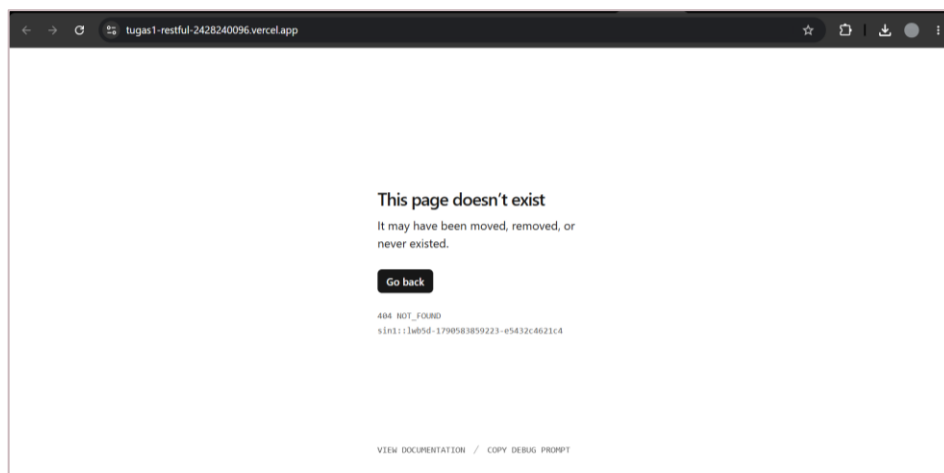
Response GET mengembalikan data langsung (objek atau array), sedangkan POST, PUT, DELETE, dan semua error memakai struktur { status, message, data } dengan data: null saat error atau setelah penghapusan. Seluruh response, termasuk 404 dan error parsing JSON, berformat JSON; tidak ada res.send() teks/HTML atau res.render().

4. Deployment

Aplikasi berhasil di-deploy ke Vercel menggunakan `vercel.json` dan ekspor `module.exports = app`. Karena data disimpan dalam array di memori dan Vercel berjalan secara serverless, data dapat kembali ke data awal setelah beberapa saat. Hal ini wajar dan tidak mengurangi kebenaran response pada setiap request.

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	Saat URL deployment pertama kali dibuka di browser, muncul halaman "404: NOT_FOUND" dari Vercel (Gambar 14), bukan JSON dari aplikasi.	Memastikan <code>app.js</code> , <code>package.json</code> , dan <code>vercel.json</code> berada di root repository, memastikan <code>vercel.json</code> (build <code>@vercel/node</code> dan route ke <code>app.js</code>) sudah di-push, memastikan deployment berstatus Ready/Production, lalu membuka domain utama project. Setelah deployment terbaru selesai, endpoint dapat diakses (Gambar 13).
2	Pengujian POST, PUT, dan DELETE tidak dapat dilakukan lewat browser karena browser hanya mengirim GET, dan halaman hasil deployment tidak memiliki tombol Send.	Pengujian dilakukan melalui Postman: memilih method, mengisi URL, dan untuk POST/PUT memilih Body → raw → JSON, lalu menekan tombol Send.



Gambar 14. Halaman 404 NOT_FOUND dari Vercel sebelum deployment dapat diakses
Halaman ini berasal dari Vercel (bukan dari aplikasi Express), sehingga penyebabnya ada pada konfigurasi deployment, bukan pada kode route.

VII. Kesimpulan

Berdasarkan praktikum, RESTful API mata kuliah pada endpoint /courses berhasil dibuat dengan Express.js dalam satu file app.js, hanya mengembalikan response JSON tanpa tampilan dan tanpa database (data disimpan dalam array di memori), sesuai tujuan 1. Enam endpoint (GET semua data, GET satu data, GET dengan filter semester, POST, PUT, dan DELETE) diimplementasikan dan diuji dengan Postman, dengan status code 200, 201, 400, dan 404 sesuai yang diharapkan (tujuan 2 dan 3).

Validasi field wajib, format response { status, message, data }, dan middleware catch-all 404 membuat API memberikan respons yang konsisten, baik saat berhasil maupun saat terjadi kesalahan (tujuan 3). Kode dikelola secara bertahap dengan Git, disimpan di repository GitHub publik, dan berhasil di-deploy ke Vercel sehingga dapat diakses melalui URL publik (tujuan 4).

Daftar Pustaka

- Express. (n.d.). *Express – Node.js web application framework*. <https://expressjs.com/>
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Fielding, R., Nottingham, M., & Reschke, J. (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9110>
- MDN Web Docs. (n.d.). *HTTP response status codes*. Mozilla. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
- Postman. (n.d.). *Postman documentation*. <https://learning.postman.com/docs/>
- Vercel. (n.d.). *Vercel documentation*. <https://vercel.com/docs>

Lampiran

Tautan Repository GitHub:

<https://github.com/nataliasiddharta/tugas1-restful-2428240096>

Tautan Deployment (jika ada):

<https://tugas1-restful-2428240096.vercel.app/courses>